Assignment 3

Transformers for Retrieval and Translation

Deadline: November 24, 2026 at 10:00 PM Eastern Time. Total: 100 points.

Instructions

Teams: Work individually or in teams of up to four. Working individually does not earn extra credit. Add all members to the submission.

Submission: Submit one `hw3.ipynb` per team on Gradescope. Complete the `# GRADED` cells and keep the supplied constructors, names and signatures. Save before running `check()`, which loads your saved code and answers and checks their formats.

Setup: Use Python 3.12, CPU PyTorch 2.14.1, NumPy 2.3.3 and matplotlib. Run `setup.sh` and select the CS440 HW3 kernel. Data and helpers are included. Use the supplied `BookAttention`. PyTorch Linear, Embedding and LayerNorm layers are allowed.

Late work: Late assignments receive 0 points.

Collusion, Plagiarism, etc.: Each team must prepare its solutions independently from other teams, i.e., without using common notes, code or worksheets with other students or trying to solve problems in collaboration with other teams. You must indicate any external sources you have used in the preparation of your solution. Do not plagiarize online sources and in general make sure you do not violate any of the academic standards of the department or the university. Failure to follow these rules may result in failure in the course.

Transformer reference

Token IDs have shape (B, T). Padding is ID 0. Unknown words use ID 1 and count as valid tokens. Learned token and position vectors produce (B, T, D) embeddings. Each of H attention heads has width D. The supplied attention projects $D \to HD$ for Q, K and V, scales scores by $\sqrt{D}$, and projects the concatenated values $HD \to D$.

Masks are boolean with *True* = *allowed*. Their final two axes are query and key positions. A $(B, 1, T)$ mask broadcasts over queries. Fully blocked rows give zero attention weights and output.

Each residual uses *postnormalization*, with LayerNorm epsilon 10^{-3}. For an encoder,

$$U = \mathrm{LN}_1(X + \mathrm{Attention}(X, X, X)), \qquad Y = \mathrm{LN}_2(U + \mathrm{FF}(U)).$$

The supplied FF uses Linear, ReLU, then Linear returning width D. The decoder adds source cross attention between self attention and FF. Useful operations include `torch.arange`, broadcasting, `torch.where`, `.sum`, `.norm`, `.gather` and `.clamp_min`.

Reading

François Chollet, *Deep Learning with Python*, second edition, Chapter 11. Read sections 11.2, 11.4 and 11.5.3. Figure 11.7 (p.341) motivates retrieval. Listings 11.21, 11.24 and 11.33 to 11.39 cover the encoder, positions and translation. Companion code is MIT licensed.

Data credits. Open Images V7 annotations are CC BY 4.0. Photos are CC BY 2.0 with individual credits in `catalog.json`. Translation text is from ManyThings and Tatoeba contributors, CC BY 2.0 FR. Full licenses accompany the data.

Problem 1 (50 points): Image retrieval

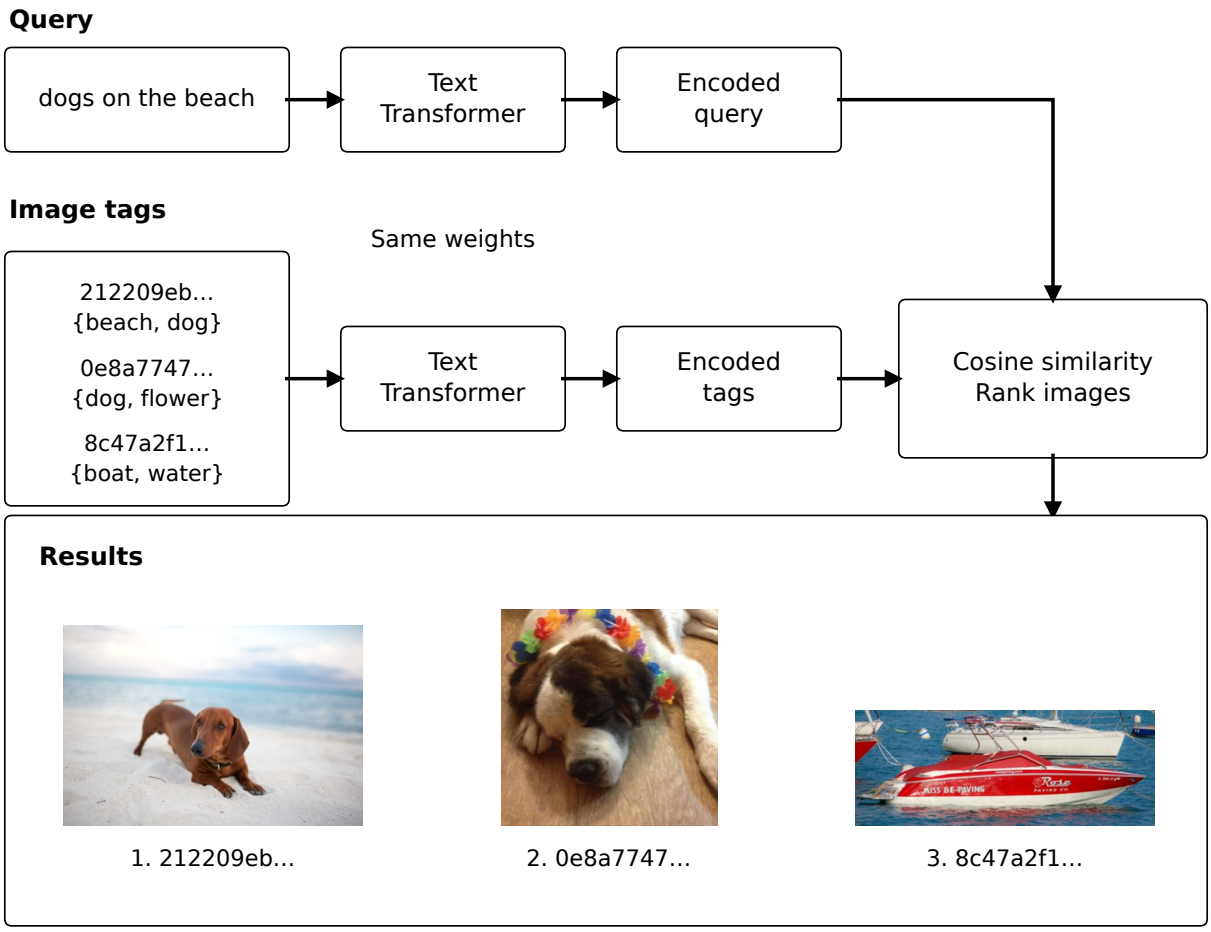


Figure 1: Text query image retrieval. Example ranking. Photos (CC BY 2.0, resized): ToddonFlickr, "0174", buyalex, "Piccard got lei'd and now he's tired.", and pulaw, "_MG_0371". Full records are in `catalog.json`.

The same text Transformer encodes a query and each image's tag set. Cosine similarity ranks the text vectors. The display helper uses ranked IDs to look up photos. The catalog contains 53 photos and 12 positive tag classes. A photo may show objects that are not in its tag list. Relevance is defined by its published keys.

The supplied preprocessing sorts and deduplicates tags before tokenization. Query order is retained. It lowercases, removes punctuation and stopwords, and maps listed plurals to singulars, such as dogs to dog and beaches to beach. See `data.py` for the complete map.

Problem 1: Requirements

(a) **Positions (8 points).** Complete `PositionalEmbedding.forward` (6) and `compute_mask` (2). Add token vectors and learned position vectors for slots 0 to $T-1$. Return shape (B, T, D) and a boolean mask IDs $\neq 0$. Support lengths $1 \leq T \leq \text{sequence_length}$.

(b) **Encoder (12 points).** Complete `TransformerEncoder.forward` using the reference equations: attention, residual and first normalization (6), FF, residual and second normalization (4), and mask handling (2). The input mask is (B, T) with True for valid keys. None permits every key.

(c) **Retriever (10 points).** Complete `TextRetriever.forward`: embedding and encoder (3), masked mean (4), normalization and all padding output (3). With valid token mask m_t and encoder outputs h_t, return (B, D) vectors

$$u = \frac{\sum_t m_t h_t}{\max(\sum_t m_t, 1)}, \qquad z = \frac{u}{\max(\|u\|_2, 10^{-8})}.$$

All padding input returns zero. ID 1 contributes normally. Trailing padding must not change the output within floating point tolerance.

(d) **Ranking (10 points).** Complete `retrieve_topk`: cosine scores (5) and ordering (5). Inputs are finite vectors (Q, D) and (N, D), unique string image IDs, and $1 \leq k \leq N$. Normalize by $\max(\|x\|_2, 10^{-8})$. Return catalog *indices and scores*, both (Q, k). Sort by decreasing score, then increasing lexicographic ID on exact ties. A zero vector has cosine 0. Do not round scores or modify inputs. NumPy is allowed.

(e) Questions (10 points)

Use the named notebook variables. Choose one letter for each multiple choice item.

1. **Figure style match scores (3 points, `Q1_FIGURE_SCORES`).** Normalize "dogs on the beach" to concepts $\{\text{dog}, \text{beach}\}$. For this item only, define relevance as the fraction of query concepts in an image's tag set, without dividing by its number of tags. Images A, B and C have sets $\{\text{beach}, \text{tree}, \text{boat}\}$, $\{\text{beach}, \text{dog}, \text{tree}\}$ and $\{\text{dog}\}$ respectively. Give the three scores in A,B,C order. Each entry is worth 1 point, absolute tolerance 10^{-6}.

2. **Tie (1 point, `Q1_TIED_IDS`).** List the two tied image IDs from the previous item in the order used by our ranking rule. Use a list of two strings.

3. **Normalization (2 points, `Q1_PROBABILITIES`).** Are the three illustrative figure style scores already attention probabilities? (A) Yes, because each is at most 1. (B) No. They do not sum to 1, whereas attention softmax weights over allowed keys do. (C) Yes, because cosine always sums to 1. (D) No, because attention weights must be negative.

4. **Image payload (2 points, `Q1_PAYLOAD`).** Which statement describes this homework? (A) The model must classify the pixels using a CNN. (B) The model averages the returned image pixels. (C) The Transformer encodes query and tag text, and a ranked image ID looks up a photograph. (D) Images supply the Spanish target tokens.

5. **Synonyms (2 points, `Q1_SYNONYM`).** A query says "puppy", but catalog keys say "dog". Which is justified? (A) Lowercasing alone guarantees the match. (B) Learned text representations may align them if supported by training, but exact tag overlap alone does not establish this. (C) Every unseen word is a padding token. (D) Figure 11.7 proves all synonyms have identical vectors.

Problem 2 (50 points): English to Spanish translation

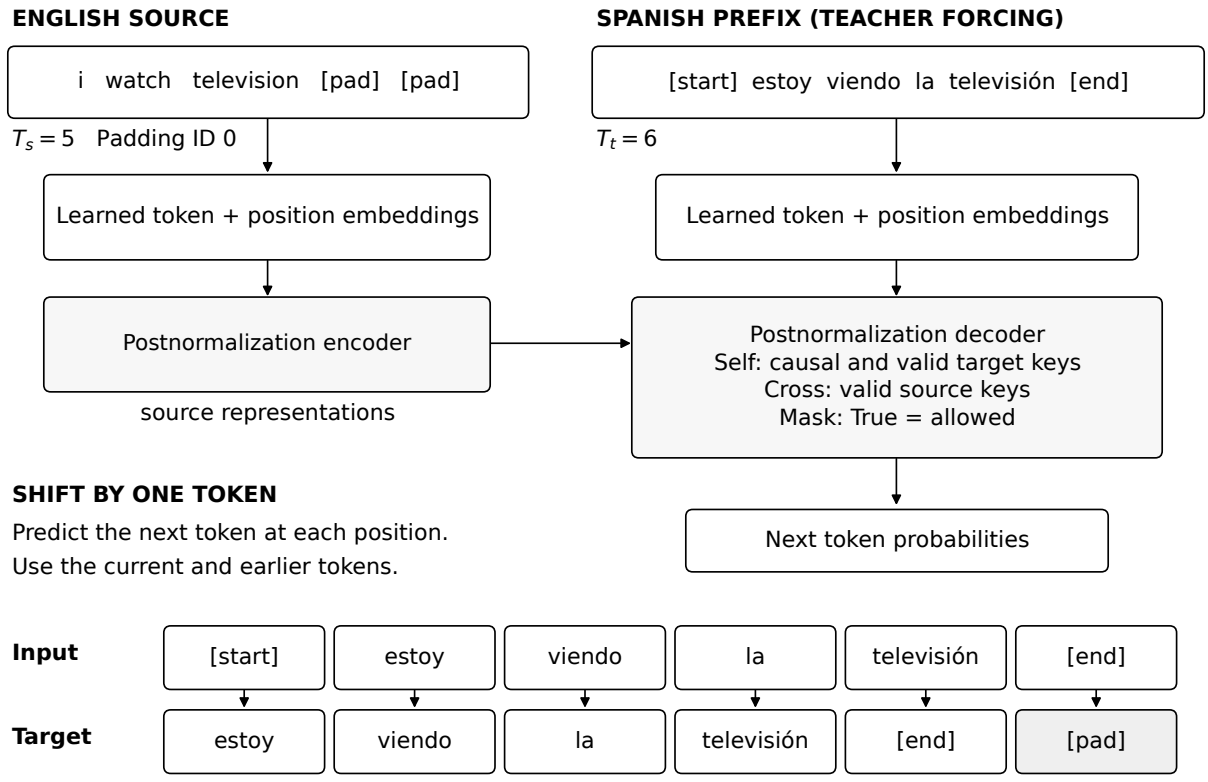


Figure 2: Teacher forcing for the validation pair "I watch television." → "Estoy viendo la televisión." ManyThings and Tatoeba contributors, CC BY 2.0 FR. Shift by one token to form targets. Exclude padding targets from metrics.

The supplied models use learned positions, width 32, two heads of width 32 each, and FF width 64. The data has 2,100 training, 450 validation and 450 test pairs. Vocabularies use training text only. Standardization keeps Spanish accents and `[start]` and `[end]`.

(a) Masks (10 points)

Complete `build_attention_masks` for long integer source IDs (B, T_s) and decoder IDs (B, T_t). Return boolean masks with True = allowed.

Mask	Shape	Allowed key j for query i	Points
Encoder	$(B, 1, T_s)$	Source ID $j \neq 0$	2
Self attention	(B, T_t, T_t)	Target ID $j \neq 0$ and $j \leq i$	4
Cross attention	(B, T_t, T_s)	Source ID $j \neq 0$	4

Lengths may differ. Every source has a valid token. Query padding rows may be zeroed. Loss excludes padding targets.

Problem 2: Requirements

(b) Decoder (15 points)

Complete `TransformerDecoder.forward`. Apply self attention, residual and `layernorm_1` (5), cross attention, residual and `layernorm_2` (5), then FF, residual and `layernorm_3` (5). Cross attention queries use the first sublayer output. Keys and values use encoder outputs. Use the supplied masks. Either may be None. Keep `get_causal_attention_mask`.

(c) Masked metrics (10 points)

Complete `masked_token_metrics(targets, probabilities)`. Targets are long IDs (B, T) and probabilities are finite normalized rows (B, T, V). Let $m_{bt} = 1[y_{bt} \neq 0]$ and $\hat{y}_{bt} = \arg\max_v p_{btv}$. Return scalar float tensors

$$L = \frac{\sum_{bt} -\log(\text{clip}(p_{bt,y_{bt}}, 10^{-7}, 1 - 10^{-7}))m_{bt}}{\sum_{bt} m_{bt}}, \qquad A = \frac{\sum_{bt} 1[\hat{y}_{bt} = y_{bt}]m_{bt}}{\sum_{bt} m_{bt}}.$$

Loss and accuracy are worth 5 points each. All padding targets give $(0, 0)$. Loss must retain gradients, including finite backward gradients for all padding targets.

(d) Greedy decoding (10 points)

Complete `greedy_decode(model, source, start_id, end_id, max_length)` for one CPU source of shape $(1, T_s)$.

1. Create a long integer zero buffer $(1, L)$ with start ID in slot 0, where $L = \max_\text{length} \geq 1$. Start and end IDs are distinct positive IDs.
2. At step i, call `model(source, buffer)` in eval mode under `torch.no_grad()`. It returns probabilities $(1, L, V)$. Choose argmax at $[0, i]$. Lowest ID breaks ties.
3. Stop on end ID or padding ID 0 without appending it. Otherwise append the token and place it in slot $i + 1$ if space remains. Make at most L predictions.
4. Return a Python list of integers excluding initial start, terminating end and padding. A later emitted start is an ordinary token. Preserve source inputs and parameters. Restore the original training flag even on error.

Buffer and timestep handling is worth 5 points. Stopping, output, eval mode, no gradients and mode restoration are worth 5.

(e) Questions (5 points)

1. `Q2_SHIFT` (2 points): a vectorized target is $[3, 4, 7, 2, 0]$, where 3=start, 2=end. Give its four teacher forcing *target* IDs, not decoder inputs. Exact equality. Numerically equal floats are accepted.
2. `Q2_MASK` (1): which keys may a decoder's cross attention see? (A) Only source slots before the target index. (B) Target valid slots. (C) All source valid slots, independently of target index. (D) Only source padding.
3. `Q2_ACCURACY` (1): high teacher-forced next token accuracy (A) does not guarantee fluent/correct whole generated translations, because inference uses its own prefixes. (B) proves no translation bias. (C) equals BLEU by definition. (D) prevents hallucinations.
4. `Q2_POSITIONS` (1): which position representation follows the assigned book model? (A) Fixed sinusoidal vectors. (B) A learned embedding table added to token embeddings. (C) Convolutions. (D) No position information.